PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Кластеризация в Python

Содержание

- Главная идея
- Кейс урока
- Ключевые понятия
- Кластеризация и классификация
- Как выглядит работа в Python
- Визуализация кластеров
- Интерпретация результата
- Почему результат может меняться
- Итог

Когда объектов много, распределить их по группам вручную становится трудно: нужно одновременно сравнивать несколько характеристик и замечать устойчивые сочетания признаков. Кластеризация помогает автоматизировать этот поиск и увидеть структуру данных, даже если названия групп заранее неизвестны.
На предыдущем уроке кластеризация рассматривалась как поиск похожих объектов на карточках и карте признаков. Теперь та же идея переносится в Python: алгоритм сам присваивает объектам номера групп, а обучающиеся строят визуализацию и объясняют смысл найденных кластеров.
Главная задача практикума — не только получить столбец с номерами кластеров, но и связать результат работы алгоритма с исходными характеристиками объектов.

Главная идея

Кластеризация ищет группы похожих объектов по признакам. В отличие от классификации, здесь нет заранее заданного правильного ответа y. Алгоритм получает только признаки X и возвращает номер группы для каждого объекта.

Сходство определяется по значениям выбранных признаков. Если два объекта имеют близкие значения speed и power, на карте этих признаков они окажутся рядом. Если значения сильно различаются, расстояние между точками будет больше.
KMeans ищет центры заданного количества кластеров и относит каждый объект к ближайшему центру. Такой центр описывает типичное положение объектов своей группы, но не обязательно совпадает с одним из реальных объектов датасета.
Важно различать результат алгоритма и его объяснение: KMeans создаёт группы по числам, а смысловые названия вроде «быстрые» или «защитники» появляются только после анализа признаков.

Кейс урока

В практикуме используется датасет игровых персонажей. У каждого персонажа есть числовые характеристики: скорость, сила и защита. Алгоритм KMeans группирует персонажей по сходству. Затем обучающиеся объясняют, какие группы получились: быстрые, сильные, сбалансированные или другие.

character	speed	power	defense
Rys	9	3	4
Grom	2	9	8
Klinok	6	7	5
Medik	4	4	8

Каждая строка описывает одного персонажа, а каждый числовой столбец — отдельное измерение для сравнения. Например, Rys заметно превосходит Grom по скорости, но уступает ему по силе и защите. Такие различия и сочетания характеристик позволяют алгоритму разделять объекты.
Столбец character содержит имена, а не числовые характеристики. Он помогает распознавать персонажей, но непосредственно в вычислении расстояний в данном примере не участвует.

Ключевые понятия

- объект - отдельная строка таблицы, например один персонаж;
- признак - числовая характеристика объекта, например speed или power;
- кластер - группа объектов, похожих по признакам;

- центр кластера - точка, относительно которой KMeans собирает близкие объекты;
- KMeans - алгоритм кластеризации, который делит объекты на заданное количество групп;
- n_clusters - число групп, которое мы просим найти;
- fit_predict() - команда, которая запускает алгоритм и возвращает номер кластера для каждого объекта.

Номер кластера — это техническая метка. Например, кластер 0 не является «лучше» или «важнее» кластера 1. Числа нужны только для различения найденных групп.

Кластеризация и классификация

В классификации модель обучается на правильных ответах: есть признаки X и известные классы y. В кластеризации столбца y нет. Алгоритм не знает заранее, какие группы должны получиться. Он смотрит на расстояния между объектами по признакам и объединяет похожие объекты.
Например, в задаче классификации для каждого персонажа мог бы быть заранее указан класс «разведчик», «боец» или «защитник». В задаче кластеризации таких ответов нет: алгоритм получает только значения характеристик и самостоятельно обнаруживает группы.
Поэтому кластер нельзя автоматически считать настоящим классом объекта. Найденное разбиение — один из способов описать структуру данных, зависящий от выбранных признаков и количества кластеров.

Как выглядит работа в Python

Сначала необходимо загрузить таблицу и отделить признаки, по которым алгоритм будет сравнивать персонажей. В примере для кластеризации выбраны только speed и power.

```
import pandas as pd

from sklearn.cluster import KMeans

df = pd.read_csv('game_characters_sklearn.csv')

X = df[['speed', 'power']]

model = KMeans(n_clusters=3, random_state=42, n_init=10)

df['cluster'] = model.fit_predict(X)

df.head()
```

Переменная df хранит исходную таблицу, а X — только те столбцы, которые передаются модели. Параметр n_clusters=3 задаёт поиск трёх групп. Параметр random_state=42 фиксирует случайную часть работы алгоритма, чтобы при повторных запусках эксперимента было проще воспроизвести результат. Параметр n_init=10 задаёт несколько запусков KMeans с разными начальными положениями центров, после чего выбирается более подходящий вариант разбиения.
Метод fit_predict(X) выполняет два действия: находит кластеры по данным X и возвращает метку для каждой строки. Количество полученных меток совпадает с количеством объектов, поэтому результат можно записать в новый столбец таблицы.
В результате появляется новый столбец cluster. В нём указаны номера найденных групп. Эти номера не являются готовыми названиями. Человек должен посмотреть на признаки объектов внутри каждого кластера и объяснить смысл группы.
В данном коде defense не входит в X, поэтому защита не влияет на распределение персонажей по кластерам. Если добавить этот столбец в X, алгоритм будет учитывать уже три характеристики, и разбиение может измениться.

Визуализация кластеров

Таблица показывает точные значения, но по ней не всегда легко заметить расположение групп. Диаграмма рассеяния переносит каждый объект на плоскость: одна координата отвечает за speed, а другая — за power.

```
import seaborn as sns

import matplotlib.pyplot as plt

sns.scatterplot(

data=df,

x='speed',

y='power',

hue='cluster'

)

plt.show()
```

На графике каждая точка - персонаж. Цвет показывает найденный кластер. Если точки одного цвета находятся рядом, значит алгоритм посчитал их похожими по выбранным признакам. Положение точки определяется значениями speed и power: чем правее находится персонаж, тем выше его скорость; чем выше точка, тем больше его сила. Параметр hue='cluster' назначает разные цвета разным номерам кластеров.
По такому двумерному графику удобно проверять именно те признаки, которые указаны на осях. Значение defense на нём не отображается, поэтому делать выводы о защите только по положению точки нельзя.
Мини-проверка: где на графике должен находиться персонаж с высокой speed и низкой power? Он окажется ближе к правой нижней части области построения.

Интерпретация результата

Одного цвета точек недостаточно, чтобы понять содержание группы. Для интерпретации нужно сравнить значения признаков внутри каждого кластера. Один из способов — вычислить средние характеристики.

```
df.groupby('cluster')[['speed', 'power', 'defense']].mean().round(1)
```

Команда groupby('cluster') собирает строки с одинаковым номером кластера, mean() вычисляет среднее значение каждого выбранного признака, а round(1) округляет результат до одного знака после запятой. В итоговой таблице одна строка соответствует одному кластеру.
Средние значения по кластерам помогают дать группам осмысленные названия. Например, если в кластере высокая speed и невысокая power, его можно назвать группой быстрых персонажей. Если высокая power и defense, но низкая speed, это может быть группа тяжёлых бойцов или защитников.
Название должно опираться на наблюдаемые значения, а не только на номер или цвет кластера. Если характеристики группы близки к средним по всем признакам, уместно название «сбалансированные персонажи». Если явного отличия нет, лучше сначала изучить отдельные строки кластера и не приписывать группе свойство без подтверждения.
При интерпретации defense важно помнить: в приведённом примере этот столбец анализируется после кластеризации, но не участвует в её построении. Поэтому высокое среднее значение defense может помочь описать уже найденную группу, однако алгоритм не использовал защиту как основание для объединения объектов.

Почему результат может меняться

Количество кластеров KMeans не определяет самостоятельно: его задают до запуска модели с помощью n_clusters. Это решение влияет на степень подробности разбиения.
Если изменить число кластеров, алгоритм разделит объекты иначе. При двух кластерах группы будут крупнее и грубее. При четырёх кластерах разбиение станет детальнее, но его может быть труднее объяснить. Поэтому в кластеризации важен не только запуск кода, но и интерпретация результата.
Результат также зависит от выбранных признаков. Кластеры по speed и power могут отличаться от кластеров по speed, power и defense, потому что в расчёте расстояний появляется дополнительная характеристика. Поэтому набор X должен соответствовать вопросу, который исследуется.
Кроме того, сами номера групп могут не нести постоянного смысла: группа быстрых персонажей в одном запуске может получить номер 0, а в другом условии — номер 2. Сравнивать нужно состав и характеристики кластеров, а не только их числовые метки.
Подходящее число кластеров выбирают так, чтобы группы были достаточно различимыми и при этом поддавались содержательному объяснению. Слишком крупное разбиение может скрыть важные различия, а слишком подробное — разделить похожие объекты на малые группы без ясного практического смысла.

Итог

К концу урока обучающиеся загружают датасет, выбирают признаки, запускают KMeans, получают номера кластеров, строят карту групп и объясняют смысл найденных кластеров через признаки объектов.
Полный ход работы можно представить как последовательность: выбрать числовые признаки X, задать количество кластеров, выполнить fit_predict(), добавить метки в таблицу, построить диаграмму и сравнить средние характеристики групп. Завершающий этап — проверить, подтверждается ли предложенное название кластера его числовыми признаками.
Ключевой вывод: алгоритм выполняет числовое разбиение объектов, но содержательная интерпретация результата остаётся задачей исследователя.

Материалы к занятию

- lesson_3_11_clustering_python_regenerated.pptx (129.8 КБ)